

Uncommon Dantzig-Wolfe Reformulation for the Temporal Knapsack Problem

Alberto Caprara, Fabio Furini, Enrico Malaguti

Dipartimento di Elettronica, Informatica e Sistemistica, University of Bologna

Viale Risorgimento, 2 - 40136 - Bologna (Italy)

e-mails: {alberto.caprara,fabio.furini,enrico.malaguti}@unibo.it

Abstract

We study a natural generalization of the knapsack problem, in which each item exists only for a given time interval. One has to select a subset of the items (as in the classical case), guaranteeing that for each time instant the set of existing selected items has total weight not larger than the knapsack capacity. We focus on the exact solution of the problem, noting that prior to our work the best method was the straightforward application of a general-purpose solver to the natural ILP formulation. Our results indicate that much better results can be obtained by using the same general-purpose solver to tackle a nonstandard Dantzig-Wolfe reformulation in which subproblems are associated with groups of constraints. This is also interesting since the more natural Dantzig-Wolfe reformulation of single constraints performs extremely poorly in practice.

1 Introduction

The well-known *Knapsack Problem* (KP), being a very basic problem in *Integer Linear Programming* (ILP), has been widely studied from all perspectives, including its practical solution to proven optimality; see, e.g., [1, 2]. The same holds for most of its natural variants and generalizations. We were surprised to find out that not much was known on the practical solution of the following generalization of KP, which we encountered in railway service design [3].

We are given a single resource and a set of n tasks, the i th having a *profit* p_i , a resource *request* w_i , and being *active* for a time interval $[s_i, t_i)$. The total resource available at any point in time is C . The problem calls for the selection of a subset of tasks of maximum total profit so that, at any point in time, the total resource required by the active tasks selected is at most C . The associated straightforward ILP formulation is the following. Clearly, it is enough to impose that the resource constraint is satisfied at the start time s_i of each task i . Letting $S_j := \{i : s_i \leq s_j \text{ and } t_i > s_j\}$ be the set of tasks active at time s_j , and x_i a binary variable

equal to 1 if task i is selected, the ILP reads:

$$\max \quad \sum_{i=1}^n p_i x_i, \quad (1)$$

$$\text{subject to} \quad \sum_{i \in S_j} w_i x_i \leq C, \quad j = 1, \dots, n, \quad (2)$$

$$x_i \in \{0, 1\}, \quad i = 1, \dots, n. \quad (3)$$

This problem has been referred to in the literature as *temporal KP* [4], *resource allocation* [5], *bandwidth allocation* [6], or *unsplittable flow on a line* [7]. The KP arises as a special case when all tasks are active for the same time interval, or, to put it differently, the problem we consider is the extension of KP in which tasks occupy the knapsack capacity only during the time interval in which they are active. Among the names above we prefer, and we will use in this paper, *Temporal KP* (TKP), since it indicates the relationship with KP.

The last one of the names above is due to the fact that the time dimension may in fact be a (one-dimensional) spatial dimension, as it is the case in our railway application. In this case, we are given a railway corridor with m stations $1, \dots, m$ and n railcars, the i th having weight w_i and to be transported from station s_i to station f_i , gaining profit p_i in this case. Moreover, we have a train that travels from station 1 to station m and can carry railcars simultaneously of a total weight of up to C . The problem is to decide which railcars the train carries in order to maximize the profit, and is clearly a TKP.

The TKP was in fact only a part of the much more complex railway design problem that we solved in [3], and the TKP instances resulting from the overall instances were fairly easy to solve. On the other hand, we decided to produce larger and more challenging instances and to try alternative solution methods on them. We did this also in light of the well-known observation that for the classical KP there are methods that widely outperform the textbook approach of solving the natural ILP formulation with a general-purpose solver.

The surprising outcome of our experiments was that the best method, among those we tried, to solve our TKP instances is based on an uncommon way to apply the *Dantzig-Wolfe Reformulation* (DWR) to ILP (1)-(3). For general references on DWR see, e.g., [8, 9]. The most natural form of DWR for this ILP, though not entirely standard, is to convexify each constraint (2) separately. The associated slave (or column generation) problem is a classical KP and can be solved within very short time. This is an indirect way to impose all valid inequalities associated with single knapsack constraints, as opposed to heuristically separate only a subset of them as every advanced general-purpose solver would do. Our preliminary computational experiments with this approach indicated that the *Linear Programming* (LP) relaxation was too slow to be solved to compensate the LP upper bound improvement due to the convexification. Accordingly, the use of DWR for TKP did not appear promising.

However this changed once we attempted to convexify not single but *multiple* constraints, i.e., to partition the set of constraints into groups and to solve a slave/column generation problem for

each group. For TKP it seems natural to put into groups constraints associated with consecutive points in time. Along with the fact that it also seems natural to form groups of (about) the same size, say k constraints per group, the unique real degree of freedom remains the value of k . The resulting slave/column generation is nothing but a smaller TKP which considers only the tasks that are active in a given time window. When k is not very small, the best way to tackle it is to use a general-purpose ILP solver.

Our finding was that the time required to solve the LP relaxation of the reformulated problem was not, as we initially expected, an increasing function of k , but in fact a *unimodal* function of k . In other words, the time required for the LP relaxation is first *decreasing* and then increasing as k increases. Jointly with the fact that the integrality gap is decreasing with k (this is guaranteed if we double k and form new groups by merging pairs of previous groups), this implies that there is an optimal trade-off value of k which is indeed fairly large, around a hundred for our instances. For a choice of k close to this value, a rudimentary depth-first branch-and-price algorithm applied to the reformulation outperforms a state-of-the-art ILP solver applied to the original ILP formulation. (The same state-of-the-art ILP solver is in fact used for the slave/column generation problems in our method.)

These results inspired the application of the same idea to generic (mixed-)ILPs discussed in [10], where part of the results of this work are reported. In this case, there are two fundamental differences with respect to TKP. First, there is the major issue to decide how to form groups of constraints, possibly leaving some of the constraints in the master problem. Second, the outcome is that the LP relaxations yield good bounds but generally a branch-and-price algorithm based on them is not competitive in terms of time to prove optimality. This makes the focus of [10] entirely different from the one of the present paper.

A subset of the instances considered in this paper were also used in [11] to test a generic solver for Dantzig-Wolfe reformulation. In that paper, however, the focus is on the effective integration of the generic solver within the general purpose open-source solver SCIP [12], without discussing how to form groups of constraints.

This paper is organized as follows. We conclude the introduction with a short literature review on TKP. Then in Section 2 we illustrate the use of DWR for the problem. In Section 3 we present the three methods we considered to find a provably optimal solution, and in Section 4 we report the associated computational results.

It is elementary to observe that everything in the paper also applies to what may be called *temporal ILPs*, which are defined starting from a regular “static” ILP and letting each variable x_i be active for a time interval $[s_i, t_i]$. A feasible solution is an assignment of values $\bar{x}_i$ to each variable x_i such that, at any point in time, setting each variable x_i to $\bar{x}_i$ if it is active, and to 0 otherwise, yields a feasible solution of the associated “static” ILP. We restrict attention to TKP here since it is the simplest case of temporal ILP and also, to the best of our knowledge, the only one that has been studied in the optimization literature.

1.1 Literature review

Almost all papers on TKP in the literature addressed theoretical aspects of the problem. However, papers on the practical solution of generalizations of the problem also exist.

Theoretical results

Arkin and Silverberg [13] consider the problem of scheduling jobs with predefined starting and ending times on C identical machines, by maximizing the value of the completed jobs, where each machine can process one job at a time. This corresponds to a TKP in which all the requests are identical: in this case the problem is trivially polynomially solvable because the constraint matrix of the ILP model (1)–(3) is totally unimodular. The authors give an $O(n^2 \log n)$ algorithm. They also prove that the variant in which each job can be processed only by a subset of machines is NP-hard, and they give an $O(n^{C+1})$ algorithm for the case of a fixed number of machines C .

Chen et al. [6] sketch a dynamic programming approach which is closely related with the $O(n^{C+1})$ algorithm in [13]. In addition, they propose a polynomial-time approximation algorithm for the special case of TKP in which the profit of a task i is proportional to $(t_i - s_i)w_i$.

Calinescu et al. [5] show a randomized polynomial-time approximation algorithm for TKP with guarantee arbitrarily close to 2.

Bansal et al. [7] show that the restricted case of TKP in which the task requests and the amount of resource available are integers in $[0, L]$ and $L \leq 2^{\text{polylog}(n)}$ has a polynomial-time approximation scheme. The results holds for the generalization of TKP in which the resource available is not necessarily a constant C but may vary over time.

Darmann et al. [14] discuss some generalizations of the problem, review complexity results for special cases of TKP, and prove that the problem remains NP-hard for the case in which all the tasks' profits are identical. In addition, they give a deterministic polynomial-time approximation algorithm with guarantee arbitrarily close to 2 for the special case in which the tasks' start and end times define a proper interval graph (i.e., no interval of activity of a task is contained in another interval of activity).

Finally, the *strong* NP-hardness of TKP was proved by Bonsma et al. [15], answering the most relevant open question on the problem complexity. The existence of a polynomial-time approximation scheme remains open.

Practical solution

The only paper on the practical solution of TKP in the literature appears to be the one of Barlett et al. [4], who propose an algorithm which integrates tree search techniques with cut generation. The main feature of the approach is to recursively decompose the problem into independent subproblems, performing a branch-and-bound search on an and-or tree. The algorithm is experimentally tested on instances with up to 600 tasks and compared with the solution of model (1)–(3) by means of the general-purpose solver CPLEX 8.1, concluding that the latter is much faster in practice. (In the paper, the authors discuss the generalization of

TKP in which the total resource available is not a constant C but varies over time, but this is not the case for the instances they solve.) These instances are fairly small compared to the ones that we can solve with our method, which are one order of magnitude larger, namely, contain up to about 9000 tasks.

There are other papers in the literature considering generalizations of TKP, which appear to be much harder to solve – compare the sizes mentioned below with those mentioned in the previous paragraph – and not suited for our DWR approach. Hall and Magazine [16] consider a generalization of TKP in which the starting time of each task can vary in a specified interval (its duration being fixed as in TKP). This models the scheduling of tasks during a space mission, where each task can take place only during a specified time interval. They propose some heuristic algorithms and upper bounds, which are integrated in an exact approach, and tested on a set of randomly generated instances with 50 to 200 tasks. The proposed exact approach can consistently solve instances with up to 50 tasks. In addition, the most famous generalization of TKP is the *multidimensional KP*, which is nothing but a 0-1 ILP with positive coefficients and “ $\leq$ ” constraints, and contains extremely difficult problems such as *set packing* as special cases. Reviewing the entire state of the art on multidimensional KP is out of the scope of this paper. We simply mention that the current best way to solve the instances in the literature appears to be the application of a general-purpose solver, possibly driving the branch-and-bound search in a proper way, see Puchinger et al. [17], leading to the optimal solution for up to 250 variables and 5 constraints.

2 Dantzig-Wolfe Reformulations

First of all, let us slightly change the ILP formulation (1)–(3) in order to get rid of dominated constraints (2) associated with a set S_j such that $S_j \subset S_k$ for some index k . Letting $N := \{j : S_j \setminus S_k \neq \emptyset \text{ for all } k \neq j\}$ be the set of indices of the non-dominated constraints, we will work with the ILP:

$$\max \quad \sum_{i=1}^n p_i x_i, \tag{4}$$

$$\text{subject to} \quad \sum_{i \in S_j} w_i x_i \leq C, \quad j \in N, \tag{5}$$

$$x_i \in \{0, 1\}, \quad i = 1, \dots, n. \tag{6}$$

The textbook approach to TKP is to solve ILP (4)–(6) directly with a general-purpose solver, possibly relying on the latter for the addition of valid inequalities to strengthen the bounds. All other approaches that come to mind to a practitioner are tentative generalizations of the solution methods for the classical KP. In this respect, even considering just the DWR of single constraints (2) appears to be an original way to approach the problem, though certainly based on an extremely well-known tool.

2.1 DWR of single constraints

Let

$$\mathcal{P}_j := \text{conv} \left\{ x \in \{0, 1\}^{S_j} : \sum_{i \in S_j} w_i x_i \leq C \right\}$$

denote the convex hull of the feasible 0-1 solutions to constraint j in (5), restricted to the space of the variables with positive coefficient in the constraint. Moreover, let $\mathcal{V}_j$ denote the set of vertices of $\mathcal{P}_j$, and, for convenience, for $v \in \mathcal{V}_j$, let v_i denote the component of v associated with task i . The DWR of constraint j in (5) amounts to imposing that the projection of x onto the components in S_j is a convex combination of the vertices in $\mathcal{V}_j$. This is achieved by introducing variables y_v^j for $v \in \mathcal{V}_j$ denoting the coefficients in the convex combination. Applying this to all constraints in (5), which are then removed, yields the reformulation:

$$\max \quad \sum_{i=1}^n p_i x_i, \tag{7}$$

$$\text{subject to} \quad x_i = \sum_{v \in \mathcal{V}_j} v_i y_v^j, \quad j \in N, \quad i \in S_j, \tag{8}$$

$$\sum_{v \in \mathcal{V}_j} y_v^j = 1, \quad j \in N, \tag{9}$$

$$x_i \in \{0, 1\}, \quad i = 1, \dots, n, \tag{10}$$

$$y_v^j \geq 0, \quad j \in N, \quad v \in \mathcal{V}_j, \tag{11}$$

where the reformulated constraints have been removed, as they are not longer necessary. Note that one could also remove the original variables, by appropriately defining the profits of the new ones. However, our experiments showed that it is better to keep them, working with an *explicit master*, as it was called in [18]. The effect of the reformulation of constraint j is to implicitly impose all the linear constraints valid for $\mathcal{P}_j$ to the LP relaxation of the original formulation (4)–(6). This at the price of introducing the $O(2^{|S_j|})$ variables y_v^j .

The LP relaxation of (7)–(11) can be solved by column generation as follows. Consider its dual, letting α_i^j and β_j denote, respectively, the dual variables associated with constraints (8) and (9). The dual constraints associated with variables y_v^j are:

$$\beta_j - \sum_{i \in S_j} v_i \alpha_i^j \geq 0, \quad j \in N, v \in \mathcal{V}_j. \tag{12}$$

Given values $\bar{\alpha}_i^j$ and $\bar{\beta}_j$ for these dual variables, the separation of (12), which corresponds to the generation of y_v^j variables with positive reduced cost, calls for the optimization of a linear objective function over $\mathcal{P}_j$. This is a classical KP, that can be solved by very effective ad-hoc methods without resorting to general-purpose ILP solvers. In fact, the time required to generate columns is negligible for this approach: the bad performances anticipated in the introduction are due to the very high number of iterations required for convergence combined with the fact that the LP relaxation quickly gets slow to solve as variables are added to it.

2.2 DWR of multiple constraints

Here we come to the successful approach which is the heart of this paper. By reasoning as for single constraints, one may reformulate multiple constraints. Formally, for $Q \subseteq N$ let

$$\mathcal{P}_Q := \text{conv} \left\{ x \in \{0, 1\}^{\bigcup_{j \in Q} S_j} : \sum_{i \in S_j} w_i x_i \leq C, j \in Q \right\}$$

denote the convex hull of the feasible 0-1 solutions to the constraints in Q among (5), $\mathcal{V}_Q$ the set of vertices of $\mathcal{P}_Q$, and y_v^Q for $v \in \mathcal{V}_Q$ the coefficients in a convex combination of the vertices in $\mathcal{V}_Q$. One may apply separately the reformulation to any collection $\mathcal{Q}$ of subsets of N . Assuming for simplicity that $\mathcal{Q}$ covers N (i.e., each $j \in N$ belongs to some $Q \in \mathcal{Q}$), this leads to the overall reformulation of TKP:

$$\max \quad \sum_{i=1}^n p_i x_i, \quad (13)$$

$$\text{subject to} \quad x_i = \sum_{v \in \mathcal{V}_Q} v_i y_v^Q, \quad Q \in \mathcal{Q}, i \in \bigcup_{j \in Q} S_j, \quad (14)$$

$$\sum_{v \in \mathcal{V}_Q} y_v^Q = 1, \quad Q \in \mathcal{Q}, \quad (15)$$

$$x_i \in \{0, 1\}, \quad i = 1, \dots, n, \quad (16)$$

$$y_v^Q \geq 0, \quad Q \in \mathcal{Q}, v \in \mathcal{V}_Q. \quad (17)$$

A major difference with respect to DWR for general ILPs discussed in [10] is that for TKP all constraints have the same form and that each variable has a positive coefficient only for a consecutive set of constraints. Accordingly, for TKP it appears to be natural to (i) let $\mathcal{Q}$ be a partition of N into pairwise disjoint subsets, hereafter called *blocks*, of (ii) almost the same size, each (iii) containing a consecutive set of constraints. We have tested a few variants of this without practical improvements, although, in principle, there are pathological cases in which, even if the cardinality of all blocks is fixed, the best LP bound is obtained by violating (iii).

In a sense our method is a divide-and-conquer approach: column generation of variables y_v^Q associated with block Q calls for the optimization of a linear objective function over $\mathcal{P}_Q$, which is nothing but a smaller TKP. The difficulty of this problem tends to increase with the size $|Q|$ of the block, the case of $|Q| = 1$ being the reformulation of a single constraint discussed above, and the case of $Q = N$ being the original TKP (with a slightly different objective function). Also the strength of the LP relaxation bound tends to increase with $|Q|$. On the other hand, the number of times that column generation is called within the solution of the LP relaxation tends to decrease with $|Q|$. A clear indication of our study is that there is an “optimal” *tradeoff* value for $|Q|$ (assuming all blocks Q have the same size, see above) leading to much smaller LP solution times with respect to the two extremes $|Q| = 1$ and $Q = N$, and to tight enough LP bounds to ensure quick solution by using a very basic branch-and-price algorithm. When $|Q|$ is not very small (as is the case for the tradeoff above) the best option to solve the column generation problem appears to be the use of a general-purpose ILP solver applied to the natural

formulation (4)–(6) (with N replaced by Q).

2.3 DWR and cut generation

It is well-known and easy to check that reformulating the constraints in block Q , by introducing the y_v^Q variables and generating them, is equivalent to separating all valid inequalities for $\mathcal{P}_Q$ without introducing further variables. Formally, given values $\bar{x}_i$ for the original variables, testing if there is a valid inequality for $\mathcal{P}_Q$ violated by $(\bar{x}_i)$ is the same as testing if $(\bar{x}_i)$ is a convex combination of the vertices in $\mathcal{V}_Q$, i.e., if there is a solution (y_v^Q) to:

$$\bar{x}_i = \sum_{v \in \mathcal{V}_Q} v_i y_v^Q, \quad i \in \bigcup_{j \in Q} S_j, \quad (18)$$

$$\sum_{v \in \mathcal{V}_Q} y_v^Q = 1, \quad (19)$$

$$y_v^Q \geq 0, \quad v \in \mathcal{V}_Q. \quad (20)$$

Adding the dummy objective function $\min 0$, this system can be solved by column generation as discussed above. If it is infeasible, an optimal dual ray yields a violated inequality. This can also be interpreted as the Benders decomposition of the constraints involving the y_v^Q variables in (13)–(17).

It is then natural to check if the direct separation of valid inequalities for $\mathcal{P}_Q$ is computationally competitive with the DWR reformulation. Our computational results clearly indicate that this is absolutely not the case. We refer the reader to, e.g., [19, 20] for further details on cut generation for ILPs.

3 Exact Solution Methods

In this section we illustrate the three methods we used to solve TKP to proven optimality.

3.1 General-purpose ILP solver

The first method is the direct application of a general-purpose ILP solver to the basic formulation (4)–(6).

3.2 Branch-and-price based on DWR

As already mentioned, the exact method based on reformulation (13)–(17) is an elementary branch-and-price algorithm that solves at each node the LP relaxation by column generation, branches on the original x_i variable whose value is closest to 1 in the LP solution, and imposes the bound on x_i in all the slaves in which this variable appears. The column generation subproblems are solved by a general-purpose ILP solver with default parameters; we tried different options and tricks to speed it up without achieving any noticeable saving. The master LP initially contains all the x variables and, for each block $Q \in \mathcal{Q}$, the variable y_0^Q associated with the null vector. This means that the unique initial solution of the master LP is to select no task. No

primal heuristic is used by our method, all integer solutions are obtained by branching and solving LP relaxations.

The solution times of the LP relaxation are greatly reduced by replacing the “=” by “ $\leq$ ” in constraints (14), which leads to an equivalent formulation, also in terms of the LP relaxation, since the set of TKP solutions forms an independence system, i.e., given any feasible subset T of tasks, every $T' \subset T$ is also feasible. The main reason for the time reduction is the well-known fact that imposing nonnegativity on the dual variables associated with (14) ensures that the dual solutions are much more stable within the column generation process.

3.3 Dynamic programming

We next illustrate a simple alternative exact solution method, which turns out to be viable when the number of feasible solutions to the KP associated with constraint j among (5), neglecting the tasks not in S_j , is not too large. (Or, equivalently, when the number $|\mathcal{V}_j|$ of vertices of $\mathcal{P}_j$ as defined in Section 2.1 is not too large.) This simple approach turns out to be the best way to solve some of the instances in our testbed.

The method is a simple dynamic programming scheme that computes the maximum-profit path in a directed acyclic graph in which nodes are partitioned into layers, one for each constraint $j \in N$. The associated layer contains $O(2^{|S_j|})$ nodes, one for each vertex in $\mathcal{V}_j$, corresponding to a subset $T \subseteq S_j$ such that $\sum_{i \in T} w_i \leq C$. Arcs connect only nodes associated with consecutive layers. Specifically, letting $\nu(j)$ denote the index after j in N , there is an arc (T, U) for $T \subseteq S_j$ and $U \subseteq S_{\nu(j)}$ if $T \cap S_{\nu(j)} = U \cap S_j$, i.e., the set of tasks active in both constraints S_j and $S_{\nu(j)}$ are the same in T and U . The profit associated with arc (T, U) is given by $\sum_{i \in U \setminus S_j} p_i$, i.e., it is the sum of the profits of the tasks in U that become active in constraint $S_{\nu(j)}$.

It is easy to check that by adding a dummy source σ connected to all nodes T in the first layer with arcs (σ, T) of profit $\sum_{i \in T} p_i$, and a dummy sink τ connected to all nodes in the last layer with arcs (σ, T) of profit 0, the determination of a maximum-profit path from σ to τ yields an optimal TKP solution of the same profit, in which the tasks selected are given by the union of the subsets visited by the path. A simple implementation using a forward search, in which the graph arcs need not be stored explicitly, has time complexity $O(\sum_{j \in N} 2^{|S_j|} \cdot 2^{|S_{\nu(j)} \setminus S_j|})$ and space complexity $O(\sum_{j \in N} 2^{|S_j|})$.

It is worth mentioning that the straightforward generalization of the classical dynamic programming scheme for KP, with one state entry for the capacity occupied in each constraint in N , was never competitive for the instances we considered, having time and space complexity $O(n(C+1)^{|N|})$.

4 Computational Results

In this section we report our computational experiments with the solution methods discussed in the paper. All computational tests were conducted by using 1 core of an INTEL Core2 Duo E6550 at 2.33GHz with 2GB of RAM, under Windows XP. The general-purpose solver used

was CPLEX 12.1, with the default parameter setting, which turned out to be a good choice for the instances we considered (we tried a few other settings without improvements). In all experiments the time limit was set to one hour CPU time. Average times are always computed by including the time limits. When the time limit is reached at least once, we indicate by (TL) the number of times this happens. In the last row denoted by *avg*, with average of average results, we indicate the total (rather than average) number of time limits.

4.1 Test instances

As already mentioned in the introduction, our interest towards TKP was motivated by an application in railway service design, but the TKP instances coming from the real-world application in [3] (which are a wide simplification of the instances of the real-world problem) turn out to be elementary to solve by a general-purpose ILP solver. The same thing happens with the only TKP instances mentioned in the literature, to the best of our knowledge, which are those of [4]. We therefore decided to create new, hopefully challenging, TKP instances and to make them publicly available in the hope of stimulating further research on the topic. We created two classes of instances, one by trying to control the number of active tasks per constraint and the other by generalizing in the (apparently) most natural way the classical random test instances of KP. We used the first class to set-up our algorithms and tuning the parameters, and the second one to verify the results in a double-blind manner. All instances are publicly available at <http://www.or.deis.unibo.it/research.html>, with detailed computational results for the single instances, while in the following we report average results.

First class

The generation of the first class of instances has the following input parameters. The number $|N|$ of constraints in the formulation (4)–(6) is specified on input (and the generation guarantees that none of these constraints is dominated). The tasks are generated by considering the constraints one after the other. The number of tasks active in each constraint is uniformly distributed in $[a_{\min}, a_{\max}]$. If the constraint is not the first one, among these active tasks a percentage uniformly distributed in $[b_{\min}, b_{\max}]$ is selected uniformly from the tasks that were already active in the previous constraint, whereas the other tasks become active in the new constraint. Profits and requests are integer values uniformly distributed in $[p_{\min}, p_{\max}]$ and $[w_{\min}, w_{\max}]$, respectively.

Table 1: Parameter values used to generate the test instances in the first class.

Group	$a_{\min}$	$a_{\max}$	$b_{\min}$	$b_{\max}$	$p_{\min}$	$p_{\max}$	$w_{\min}$	$w_{\max}$	$p = w?$	$n_{\min}$	$n_{\max}$	$ N _{\min}$	$ N _{\max}$
I	10	10	90	95	10	100	10	100	no	2697	3849	2688	3840
II	15	15	90	95	10	100	10	100	no	4480	6462	2688	3840
III	20	20	90	95	10	100	10	100	no	4972	7026	2688	3840
IV	25	25	90	95	10	100	10	100	no	6322	8977	2688	3840
V	30	30	90	95	10	100	10	100	no	2071	5129	768	1920
VI	30	30	70	90	10	100	10	100	no	4948	12498	768	1920
VII	30	30	90	95	10	100	10	100	yes	2071	5142	768	1920
VIII	25	35	90	95	10	100	10	100	no	2916	7241	768	1920
IX	25	35	70	90	10	100	10	100	no	5210	13025	768	1920
X	30	40	90	95	10	100	10	100	no	3117	7709	768	1920

We considered 10 groups of instances, called I, ..., X, each with the values of the parameters as reported in Table 9, where the column $p = w?$ indicates whether profit and requests are the same (this is the case only for Group VII). The last four columns report respectively the minimum and the maximum number of variables and constraints in the formulation (4)–(6). We generated 10 instances for each group, which differ from each other by the number $|N|$ of constraints. Although instances within each group have a different size, average values reported in the computational experiments allow a meaningful comparison among different groups. For Groups I–IV, $|N| = 2688 + 128(i - 1)$ for instances $i = 1, \dots, 10$. For Groups V–X, $|N| = 768 + 128(i - 1)$ for instances $i = 1, \dots, 10$.

Second class

For the second class of instances we considered the way in which the most common class of KP instances, namely the so-called *uncorrelated problems* [1, 2], are generated. For KP, one simply has to generate p_i and w_i , which are both uniformly random in $[1, p_{\max}]$ in this case. In our case, we also have to generate the task interval $[s_i, t_i]$. To this end, we let s_i be uniformly random in $[1, s_{\max}]$ and the duration $t_i - s_i$ be uniformly random in $[1, d_{\max}]$. Finally, the capacity is set to $\alpha \sum_{i=1}^n w_i$.

In the classical KP case, typical values for $p_{\max}$ and α are 100 and 0.5, respectively. In our case, we also set $p_{\max} = 100$. In addition we set $s_{\max} = 1000$ and considered 10 groups of instances, each with $n = 1000$ tasks, with $d_{\max} \in \{10, 20, 30, 40, 50, 60, 70, 80, 90, 100\}$. The resulting ten groups are called XI, ..., XX. In order to obtain challenging instances for this choice of parameters, we set $\alpha = 0.01$ (for larger values of α , a wide fraction of the tasks can be selected in the solution, and the instances tend to be easy). By construction, all these instances have roughly the same number $|N|$ of constraints in the formulation (4)–(6), around 450.

4.2 Setting the size of the groups in DWR

First of all, a key issue to address for the approach based on DWR is the number $|Q|$ of constraints in each block. We present the effect of using values that are integer powers of 2 for all the instances in the first class, presenting average values for each of the ten Groups I–X. The results for the instances in the second class are entirely similar and not reported here. In Table 2, for $|Q| \in \{1, 2, 4, 8, 16, 32, 64, 128, 256\}$, we report the time needed to solve the LP relaxation of the DWR reformulation (13)–(17), the total time needed to solve the LP relaxation of the master problem during the procedure, the number of columns generated and the associated percentage integrality gap with respect to the optimal TKP solution value. The integrality gaps for the instances reaching the time limit refer to the Lagrangian upper bound on the optimal LP value, obtained by summing to the current master LP value the violation of the dual constraints.

The table points out that, by increasing the block size $|Q|$, the computing time needed to solve the LP relaxation initially decreases. This is due to the fact that the decreasing number of iterations of column generation compensates, up to a certain point, the fact that generating columns becomes harder. Specifically, for $|Q| = 32$ and $|Q| = 64$ the computing time is one

Table 2: Comparison of the LP relaxations of (13)-(17) solved with column generation for various values of the block size $|Q|$.

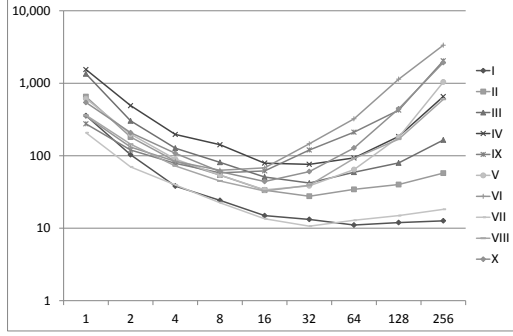
	$ Q = 1$				$ Q = 2$				$ Q = 4$			
	time	timeLP	nCols	gap	time	timeLP	nCols	gap	time	timeLP	nCols	gap
I	357.9	23.6	39469	0.99	103.2	4.6	18099	0.74	38.1	1.0	8268	0.42
II	653.0	97.4	56444	1.36	181.1	12.2	25061	0.87	84.4	2.0	10737	0.46
III	1348.5	259.1	81935	1.58	302.6	44.7	34898	1.04	127.3	5.8	14379	0.55
IV	1546.1	403.8	97651	1.85	491.8	79.3	41542	1.19	196.6	9.4	16861	0.61
V	612.0	183.4	47161	2.13	198.7	34.2	20317	1.40	89.8	3.9	8068	0.72
VI	351.4	38.9	29718	1.35	131.5	4.5	12326	0.62	83.7	1.1	4895	0.26
VII	207.3	21.2	23338	0.20	70.0	2.8	10220	0.13	39.5	0.7	4338	0.08
VIII	362.7	82.2	35121	1.43	144.2	15.4	15772	0.95	72.4	1.7	6490	0.47
IX	275.5	26.7	26725	1.08	119.8	3.8	11638	0.54	78.6	1.1	4838	0.25
X	543.6	134.4	42437	1.57	208.0	28.6	19130	1.03	108.1	3.8	7738	0.49
avg	625.8	127.1	48000	1.35	195.1	23.0	20900	0.85	91.9	3.1	8661	0.43

	$ Q = 8$				$ Q = 16$				$ Q = 32$			
	time	timeLP	nCols	gap	time	timeLP	nCols	gap	time	timeLP	nCols	gap
I	24.2	0.4	3640	0.19	14.9	0.2	1608	0.08	11.0	0.1	771	0.04
II	54.1	0.7	4495	0.21	33.4	0.3	1976	0.10	25.7	0.2	911	0.05
III	81.1	1.0	5702	0.23	50.9	0.4	2428	0.11	40.0	0.3	1064	0.06
IV	141.9	1.5	6519	0.27	79.0	0.6	2720	0.12	72.8	0.4	1265	0.07
V	54.1	0.7	3102	0.32	33.8	0.3	1278	0.12	37.9	0.2	564	0.05
VI	63.0	0.5	2100	0.11	68.0	0.3	985	0.05	147.0	0.3	485	0.03
VII	22.4	0.3	1691	0.03	13.5	0.1	751	0.02	10.7	0.1	353	0.00
VIII	44.8	0.5	2546	0.18	32.6	0.3	1077	0.09	38.9	0.1	518	0.04
IX	58.1	0.5	2080	0.10	61.7	0.3	975	0.05	119.4	0.3	477	0.02
X	60.4	0.7	3024	0.21	44.1	0.3	1253	0.08	59.9	0.2	601	0.04
avg	60.4	0.7	3490	0.19	43.2	0.3	1505	0.08	56.3	0.2	701	0.04

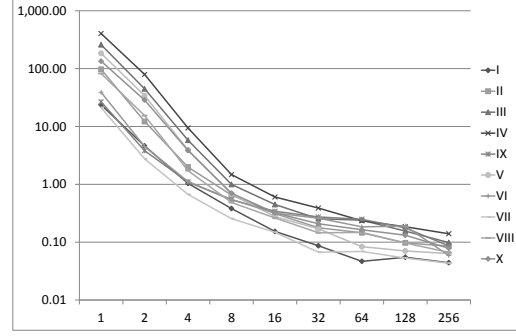
	$ Q = 64$				$ Q = 128$				$ Q = 256$			
	time	timeLP	nCols	gap	time (TL)	timeLP	nCols	gap	time (TL)	timeLP	nCols	gap
I	11.1	0.0	369	0.02	11.9 (0)	0.1	184	0.02	12.6 (0)	0.0	87	0.00
II	34.2	0.1	442	0.02	39.9 (0)	0.1	227	0.02	57.5 (0)	0.1	106	0.01
III	58.8	0.2	547	0.02	79.1 (0)	0.2	264	0.01	164.8 (0)	0.1	128	0.01
IV	92.7	0.2	606	0.04	183.8 (0)	0.2	289	0.01	656.1 (0)	0.1	146	0.00
V	64.1	0.1	257	0.02	423.7 (0)	0.1	122	0.01	2040.4 (4)	0.1	57	0.00
VI	321.7	0.2	241	0.01	179.3 (0)	0.2	121	0.01	1040.9 (0)	0.1	33	0.14
VII	12.8	0.1	172	0.00	1133.9 (1)	0.1	85	0.00	3350.9 (9)	0.0	45	0.00
VIII	91.5	0.1	252	0.02	14.8 (0)	0.1	117	0.01	18.2 (0)	0.1	49	0.00
IX	210.3	0.2	226	0.01	168.6 (0)	0.2	110	0.01	599.9 (0)	0.1	44	0.03
X	127.3	0.2	307	0.02	441.5 (0)	0.1	144	0.01	1931.8 (3)	0.1	53	0.06
avg	102.4	0.2	342	0.02	267.7 (1)	0.1	166	0.01	987.3 (16)	0.1	75	0.03

order of magnitude smaller than for $|Q| = 1$, but also one or more orders of magnitude smaller than for $|Q| = 256$. On the other hand, the integrality gap is already very small for $|Q| = 32$ and $|Q| = 64$. The advantage of reformulating multiple constraints, for a correct choice of the block size, is therefore extremely evident. Also clear is that this choice is *robust*, in that there is no magic value but a fairly wide range around the value for which the minimum LP solution time is attained. Note that solution time for the master LP is fairly small, namely the solution time is widely dominated by the time needed to generate the columns.

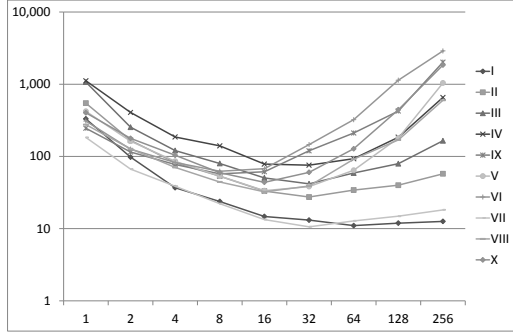
In Figure 1 we give a graphical representation of the results in Table 2, plus some additional information. Parts (a), (b), (c) and (d) are in logarithmic scale. Part (a) shows the total time to solve the LP relaxation of the DWR reformulation (13)-(17) (column *time* in Table 2). Part



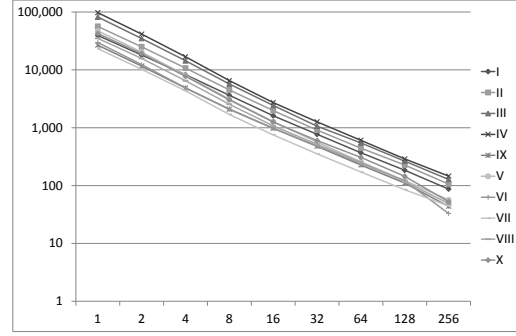
(a) *LP relaxation time*



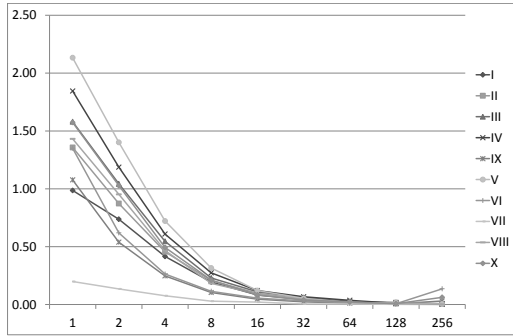
(b) *Master LP time*



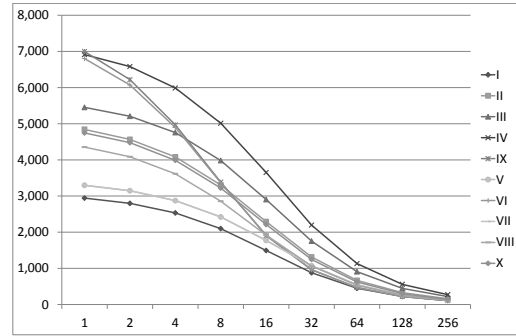
(c) *Column generation time*



(d) *Number of columns generated*



(e) *Integrality gap*



(f) *Number of x variables in more than one block*

Figure 1: Comparison of the LP relaxations of (13)-(17) solved with column generation for various values of the block size $|Q|$.

(b) shows the total time needed to solve the LP relaxation of the master problem during the procedure (*timeLP*). Part (c) shows the total time needed to solve the ILPs associated with the slave problems for column generation. Part (d) shows the number of columns generated (*nCols*). Part (e) shows the percentage integrality gap with respect to the optimal TKP solution value (*gap*). Finally, Part (f) shows the number of x variables present in more than one block, which affects the number of constraints (14).

In the following we will present results only for $|Q| \in \{32, 64, 128\}$, which are by far the three best choices for our instances (restricting attention to powers of 2). Without reporting the associated results, we let the reader imagine how bad a branch-and-price approach based on DWR of single constraints may behave in view of Table 2.

4.3 DWR versus cut generation

In Table 3, we consider the solution of the LP relaxation of the DWR reformulation (13)-(17) by the Benders decomposition approach explained in Section 2.3, which avoids the explicit handling of the y variables and separates valid inequalities for $\mathcal{P}^Q$ in the space of the original variables. As in Section 4.2, we consider only the instances in the first class, the results for the second class being analogous. For $|Q| \in \{1, 2, 4, 8, 16, 32\}$, we report, the same information as in Table 2, the number of columns being replaced by the number of generated cuts. If the time limit is not reached, the integrality gap coincides with the one in Table 2. Otherwise, the optimal value of current LP provides a valid bound.

The results clearly show the huge advantage of DWR with respect to the cut generation version, due to the fact that the separation of valid inequalities requires in itself the generation of columns. Note that in this case the average time increases with $|Q|$. For the “good” choices of Q indicated in Section 4.2, the cut generation almost always reaches the time limit, and the associated upper bound is of very bad quality.

4.4 DWR and branch-and-price versus a general-purpose solver

We now illustrate the results of the first two methods of Section 3 for instances in Groups V-X in the first class, and for the instances in the second class, all of which cannot be solved by dynamic programming, the third method of Section 3, as the average number of active tasks per constraint is too large.

For the first class, in Table 4 we compare the solution times and the integrality gaps of the following LP relaxations of TKP: the basic one, associated with the formulation (4)-(6), the one obtained at the root node by CPLEX applied to the basic formulation (with the addition of the default CPLEX cutting planes) and finally the one obtained with the reformulation (13)-(17) considering $|Q| \in \{32, 64, 128\}$ (according to Section 4.2). Table 4 clearly shows that for the block sizes $|Q|$ considered the optimality gap of DWR with respect to the CPLEX root is reduced by about two orders of magnitude at the cost of increasing the running time by one to two orders of magnitude.

Table 3: Comparison of the LP relaxations of (13)-(17) solved with cut generation for various values of the block size $|Q|$.

	$ Q = 1$				$ Q = 2$				$ Q = 4$			
	time	timeLP	nCuts	gap	time	timeLP	nCuts	gap	time	timeLP	nCuts	gap
I	146.1	0.9	13748	0.99	63.9	0.7	8545	0.74	42.3	0.6	5319	0.42
II	284.3	3.0	22057	1.36	151.3	2.3	13911	0.87	136.0	2.0	8773	0.46
III	606.4	7.4	28775	1.58	261.7	5.4	18279	1.04	252.9	4.2	11316	0.55
IV	621.2	14.3	33994	1.85	458.4	11.4	23229	1.19	522.7	8.6	14427	0.61
V	345.1	11.6	17713	2.13	248.3	7.8	11453	1.40	344.6	5.9	7148	0.72
VI	464.0	12.5	23531	1.35	692.9	10.6	15227	0.62	2528.1	11.9	10120	0.26
VII	224.9	4.6	17692	0.20	148.4	3.6	11494	0.13	163.0	2.6	7298	0.08
VIII	349.6	10.1	18716	1.43	329.6	7.9	13004	0.95	666.3	7.4	8623	0.47
IX	447.7	11.8	23252	1.08	735.2	10.5	15913	0.54	2529.3	10.8	10175	0.25
X	486.3	17.0	21393	1.57	487.7	13.8	14874	1.03	969.1	11.9	9610	0.49
avg	397.5	9.3	22087	1.35	357.7	7.4	14593	0.85	815.4	6.6	9281	0.43

	$ Q = 8$				$ Q = 16$				$ Q = 32$			
	time (TL)	timeLP	nCuts	gap	time (TL)	timeLP	nCuts	gap	time (TL)	timeLP	nCuts	gap
I	47.9 (0)	0.6	3528	0.19	130.8 (0)	0.7	2804	0.08	656.1 (0)	1.0	2621	0.04
II	266.0 (0)	2.0	6091	0.21	1170.1 (0)	2.6	5150	0.10	3600.0 (10)	1.6	3337	0.87
III	526.3 (0)	4.1	7549	0.23	2220.6 (0)	5.1	6107	0.11	3600.0 (10)	1.5	2855	1.81
IV	1426.5 (0)	8.7	9665	0.27	3600.0 (10)	5.0	5673	0.59	3600.0 (10)	0.9	2112	4.09
V	1031.5 (0)	6.3	4792	0.32	3551.0 (6)	4.9	3185	0.34	3600.0 (10)	0.6	1159	3.29
VI	3600.0 (10)	1.8	4204	1.83	3600.0 (10)	0.1	532	9.11	3600.0 (10)	0.2	410	9.77
VII	379.9 (0)	2.2	5080	0.03	1752.1 (0)	2.8	4272	0.02	3600.0 (10)	1.0	2749	0.35
VIII	2717.8 (3)	9.1	6376	0.19	3600.0 (10)	1.2	2473	1.62	3600.0 (10)	0.1	405	7.93
IX	3600.0 (10)	1.5	4280	1.75	3600.0 (10)	0.1	545	8.22	3600.0 (10)	0.1	140	10.91
X	3236.0 (4)	10.7	6326	0.27	3600.0 (10)	0.9	2118	2.26	3600.0 (10)	0.1	296	8.91
avg	1683.2 (27)	4.7	5789	0.53	2682.5 (56)	2.3	3286	2.24	3305.6 (90)	0.7	1608	4.80

Table 4: Comparison of LP relaxations of TKP for the instances in Groups V-X in the first class.

	Basic Model, LP		Basic Model, root		DWR, $ Q = 32$, LP		DWR, $ Q = 64$, LP		DWR, $ Q = 128$, LP	
	time	gap	time	gap	time	gap	time	gap	time	gap
V	0.0	15.67	2.7	2.23	37.9	0.05	64.1	0.02	179.3	0.01
VI	0.1	13.63	5.3	1.52	147.0	0.03	321.7	0.01	1133.9	0.01
VII	0.0	10.99	1.8	0.25	10.7	0.00	12.8	0.00	14.8	0.00
VIII	0.1	13.21	2.9	1.62	38.9	0.04	91.5	0.02	168.6	0.01
IX	0.1	12.41	4.6	1.23	119.4	0.02	210.3	0.01	423.7	0.01
X	0.1	13.23	3.3	1.94	59.9	0.04	127.3	0.02	441.5	0.01
avg	0.1	13.19	3.4	1.47	69.0	0.03	137.9	0.01	393.6	0.01

Table 5 shows that the considerable increase in the root node solution time from CPLEX to DWR indeed pays off when it comes to find an optimal solution by branch-and-bound, with a significant reduction in the number of branching nodes. Except from Group VI, for which the fact that profits and requests coincide makes the instances easy, CPLEX is unable to solve a single instance out of the 50 in the other groups within the one-hour time limit. On the other hand, DWR solves to optimality 42 of these instances for $|Q| = 32$, 47 for $|Q| = 64$ and 43 for $|Q| = 128$. The average running time for the instances solved by DWR is a few minutes. The optimality gaps are reported only if a method reaches the time limit. When also DWR reaches the time limit, its optimality gap is one order of magnitude smaller than for CPLEX. In the

following tables, we highlight in bold the best exact method, i.e., the one reaching the time limit the smallest number of times breaking ties by lowest average computing time.

Table 5: Comparison of exact methods for TKP for the instances in Groups V-X in the first class.

	Basic Model, B&B			DWR, $ Q = 32$, B&P			DWR, $ Q = 64$, B&P			DWR, $ Q = 128$, B&P		
	time (TL)	gap	nodes	time (TL)	gap	nodes	time (TL)	gap	nodes	time (TL)	gap	nodes
V	3600.0 (10)	0.90	414538	480.0 (0)		170	176.2 (0)		36	501.7 (0)		36
VI	3600.0 (10)	0.75	358125	1638.4 (2)	0.1	108	1154.3 (0)		37	1884.8 (3)	0.01	24
VII	5.1 (0)	0.9	520	15.3 (0)		5	17.8 (0)		6	16.0 (0)		1
VIII	3600.0 (10)	0.56	368934	1002.1 (1)	0.1	275	572.2 (0)		72	304.0 (0)		10
IX	3600.0 (10)	0.54	367807	1410.4 (3)	0.0	104	1085.6 (2)	0.03	41	812.1 (1)	0.02	9
X	3600.0 (10)	0.86	371790	1270.8 (2)	0.1	195	911.9 (1)	0.05	82	1388.5 (3)	0.02	32
avg	3000.8 (50)	0.76	313619	969.5 (8)	0.1	143	653.0 (3)	0.04	46	817.8 (7)	0.01	19

The results for the instances of the second class are reported in Tables 6 and 7, which have the same format as Tables 4 and 5, respectively. In this case, for DWR, $|Q| = 128$ is slightly better than $|Q| = 64$ and $|Q| = 32$. In addition, we restrict attention to the 50 instances in Groups XIII-XVII, as the instances in Groups XI and XII are very easy, and none of the instances in Groups XVIII-XX could be solved to proven optimality. The results reported in Table 7 show that CPLEX solves all the instances in Group XIII to proven optimality within a time that is almost an order of magnitude smaller than the time taken by DWR to solve the root node, at which it always finds the optimum except in one case. Already for Group XIV, CPLEX solves only half of the instances, whereas DWR solves all of them but one (the two unsolved instances for $|Q| = 128$ and $|Q| = 64$ have only one common element). This trend continues for Groups XV-XVII, with CPLEX reaching the time limit in all cases and DWR solving 6 instances in Group XV (4 with $|Q| = 128$ and 2 more for $|Q| = 64$), 1 instance in Group XVI, and no instance in Group XVII. In the unsolved cases the final DWR gap is generally smaller, sometimes much smaller, than the final CPLEX gap. Differently from the instances in the first class, for these DWR generally reaches the time limit already at the root node.

Table 6: Comparison of LP relaxations of TKP for the instances in the second class.

	Basic Model, LP		Basic Model, root		DWR, $ Q = 32$, LP		DWR, $ Q = 64$, LP		DWR, $ Q = 128$, LP	
	time	gap	time	gap	time (TL)	gap	time	gap	time (TL)	gap
XIII	0.0	1.79	0.3	0.24	557.2 (0)	0.01	169.7 (0)	0.00	77.8 (0)	0.00
XIV	0.0	1.88	0.4	0.45	1857.2 (3)	0.02	1194.5 (1)	0.01	1414.1 (1)	0.01
XV	0.0	1.81	0.5	0.59	3067.8 (6)	0.04	2655.9 (4)	0.03	2621.5 (5)	0.01
XVI	0.0	1.80	0.6	0.67	3563.1 (9)	0.12	3354.3 (8)	0.12	3351.1 (9)	0.15
XVII	0.0	1.78	0.6	0.76	3562.3 (9)	0.17	3600.0 (10)	0.19	3600.0 (10)	0.20
avg	0.0	1.81	0.5	0.54	2521.5 (27)	0.07	2194.9 (23)	0.07	2212.9 (25)	0.07

4.5 Dynamic programming versus all for small-support constraints

For instances in Groups I-IV in the first class, the dynamic programming method of Section 3.3 can also be applied, as the average number of active tasks per constraint is sufficiently small. The associated results are reported in Table 8.

Table 7: Comparison of exact methods for TKP for the instances in the second class.

	Basic Model, B&B			DWR, $ Q = 32$, B&P			DWR, $ Q = 64$, B&P			DWR, $ Q = 128$, B&P		
	time (TL)	gap	nodes	time (TL)	gap	nodes	time (TL)	gap	nodes	time (TL)	gap	nodes
XIII	11.2 (0)		4101	881.4 (0)		575	211.2 (0)		72	82.8 (0)		3
XIV	2228.7 (5)	0.08	639348	2852.7 (6)	0.0	792	1847.7 (2)	0.03	445	1907.4 (2)	0.04	123
XV	3600.0 (10)	0.17	666381	3600.0 (10)	0.0	202	3217.0 (7)	0.04	209	2830.2 (6)	0.02	16
XVI	3600.0 (10)	0.25	553044	3600.0 (10)	0.1	44	3493.0 (9)	0.13	28	3371.7 (9)	0.16	2
XVII	3600.0 (10)	0.34	460084	3600.0 (10)	0.2	8	3600.0 (10)	0.19	0	3600.0 (10)	0.20	0
avg	2608.0 (35)	0.21	464592	2906.8 (36)	0.1	324	2473.8 (28)	0.10	151	2358.4 (27)	0.11	29

Note that when dynamic programming reaches the time limit, there are no feasible solutions nor upper bounds available, and therefore no associated gap. Table 8 shows that dynamic programming is the best method for the instances in Groups I and II, and also for III, being comparable to DWR. On the other hand, it cannot solve any instance in Group IV. As to CPLEX, it is the second best method for Group I but the worst one for Groups II, III and IV, being unable to solve any of the instances in the last two. DWR with $|Q| = 128$ is the only method capable of solving all the 40 instances within the time limit.

Table 8: Comparison of exact methods for TKP for the instances in Groups I-IV of the first class.

	Basic Model, B&B			DWR, $ Q = 32$, B&P			DWR, $ Q = 64$, B&P			DWR, $ Q = 128$, B&P		Dyn. Prog time (TL)
	time (TL)	gap	nodes	time (TL)	gap	nodes	time (TL)	gap	nodes	time (TL)	nodes	
I	7.6 (0)		658	91.9 (0)		39	24.9 (0)		10	20.2	6	1.0 (0)
II	1988.4 (3)	0.06	182120	636.6 (0)		69	158.0 (0)		24	128.2	20	9.5 (0)
III	3600.0 (10)	0.40	354977	1972.6 (4)	0.08	216	627.2 (0)		82	284.4	22	182.2 (0)
IV	3600.0 (10)	0.93	331616	2906.6 (7)	0.07	239	1091.5 (1)	0.07	106	433.2	16	3600.0 (10)
avg	2299.0 (23)	0.46	217342	1401.9 (11)	0.08	141	475.4 (1)	0.07	55	216.54	16	948.2 (10)

5 Conclusions

Generally DWR is used when the problem at hand admits a natural decomposition, the prototype example being the well-known bin packing (or cutting stock) problem. In this paper we have shown the potential of the method in a context in which there is no natural decomposition. As already mentioned in the paper, this outcome for TKP inspired the investigation of the method for general (Mixed) ILPs, which started with [10]. However, in terms of exact solution the TKP case is strikingly successful and therefore deserves a separate treatment, given in this paper.

Our computational experiments also showed that DWR combined with column generation widely outperforms the equivalent approach based on cut generation. In passing, we also proposed a dynamic programming algorithm, which turns out the best method for solving our TKP instances with small-support constraints.

On the other hand, among the instances we proposed, three in the first class and quite a few in the second remained unsolved. We hope that this will stimulate further research on this very natural generalization of KP. The most optimistic scenario is that their study will lead to ideas

applicable in a more general setting, as it was the case for this work.

Acknowledgments

We are grateful to Marco Lübbecke for illuminating discussions on the subject, and to the associate editor and two anonymous referees for their helpful comments.

References

- [1] H. Kellerer, U. Pferschy, and D. Pisinger. *Knapsack Problems*. Springer-Verlag, Berlin, 2004.
- [2] S. Martello and P. Toth. *Knapsack Problems: Algorithms and Computer Implementations*. John Wiley & Sons, Chichester, 1990.
- [3] A. Caprara, E. Malaguti, and P. Toth. A freight service design problem for a railway corridor. *Transportation Science*, 45(2):147–162, 2011.
- [4] Y. Hamadi I. Miguel S. A. Tarim M. Bartlett, A. M. Frisch and C. Unsworth. The temporal knapsack problem and its solution. In *Proceedings of the International Conference on Integration of AI and OR Techniques in Constraint Programming for Combinatorial Optimization Problems*, pages 34–48. Springer-Verlag, 2005.
- [5] Gruia Calinescu, Amit Chakrabarti, Howard J. Karloff, and Yuval Rabani. Improved approximation algorithms for resource allocation. In *Proceedings of the 9th International IPCO Conference on Integer Programming and Combinatorial Optimization*, pages 401–414. Springer-Verlag, 2002.
- [6] B. Chen, R. Hassin, and M. Tzur. Allocation of bandwidth and storage. *IIE Transactions*, 24:501–507, 2002.
- [7] N. Bansal, A. Chakrabarti, A. Epstein, and B. Schieber. A quasi-PTAS for unsplittable flow on line graphs. In *Proceedings of the 38th Annual ACM Symposium on Theory of Computing (STOC)*, pages 721–729. Seattle, 2006.
- [8] M. Lübbecke and J. Desrosiers. Selected topics in column generation. *Operations Research*, 53(6):1007–1023, 2004.
- [9] F. Vanderbeck and L. A. Wolsey. Reformulation and decomposition of integer programs. In *50 Years of Integer Programming 1958–2008: From the Early Years to the State-of-the-Art*, pages 431–502. Springer-Verlag, Berlin, Heidelberg, 2010.
- [10] M. Bergner, A. Caprara, F. Furini, M. Lübbecke, E. Malaguti, and E. Traversi. Partial convexification of general MIPs by Dantzig-Wolfe reformulation. In *Integer Programming and Combinatorial Optimization (IPCO 2011)*, pages 39–51. Springer-Verlag, Berlin, 2011.
- [11] G. Gamrath and M. E. Lübbecke. Experiments with a generic Dantzig-Wolfe decomposition

for integer programs. In *Proceedings of the 9th International Symposium on Experimental Algorithms (SEA)*, pages 239–252. Springer-Verlag, Berlin, 2010.

- [12] SCIP. <http://scip.zib.de/>, 2011. Accessed April 2011.
- [13] E. M. Arkin and E. B. Silverberg. Scheduling jobs with fixed start and end times. *Discrete Applied Mathematics*, 18(1):1–8, 1987.
- [14] A. Darmann, U. Pferschy, and J. Schauer. Resource allocation with time intervals. *Theoretical Computer Science*, 411(49):4217–4234, 2010.
- [15] P. Bonsma, J. Schulz, and A. Wiese. A constant factor approximation algorithm for unsplittable flow on paths. *CoRR*, abs/1102.3643, 2011.
- [16] N. G. Hall and M. J. Magazine. Maximizing the value of a space mission. 78:224–241, 1994.
- [17] J. Puchinger, G. R. Raidl, and U. Pferschy. The multidimensional knapsack problem: Structure and algorithms. *INFORMS Journal on Computing*, 22(2):250–265, 2010.
- [18] M. P. de Aragão and E. Uchoa. Integer program reformulation for robust branch-and-cut-and-price algorithms. In *Proceedings of the Conference Mathematical Program in Rio: A Conference in Honour of Nelson Maculan*, pages 56–61. Rio de Janeiro, 2003.
- [19] T. K. Ralphs and M. V. Galati. Decomposition and dynamic cut generation in integer linear programming. *Mathematical Programming*, 106(2):251–285, 2006.
- [20] H. D. Sherali, Y. Lee, and Y. Kim. Partial convexification cuts for 0-1 mixed-integer programs. *European Journal of Operational Research*, 165(3):625–648, 2005.

A Detailed Characteristics of the Instances Considered

Table 9: Parameter values used to generate the test instances in the first class.

Group	Instances	$a_{\min}$	$a_{\max}$	$b_{\min}$	$b_{\max}$	$p_{\min}$	$p_{\max}$	$w_{\min}$	$w_{\max}$	$p = w?$	$n_{\min}$	$n_{\max}$	$ N _{\min}$	$ N _{\max}$
I	I1-I10	10	10	90	95	10	100	10	100	no	2697	3849	2688	3840
II	I11-I20	15	15	90	95	10	100	10	100	no	4480	6462	2688	3840
III	I21-I30	20	20	90	95	10	100	10	100	no	4972	7026	2688	3840
IV	I31-I40	25	25	90	95	10	100	10	100	no	6322	8977	2688	3840
V	I41-I50	30	30	90	95	10	100	10	100	no	2071	5129	768	1920
VI	I51-I60	30	30	70	90	10	100	10	100	no	4948	12498	768	1920
VII	I61-I70	30	30	90	95	10	100	10	100	yes	2071	5142	768	1920
VIII	I71-I80	25	35	90	95	10	100	10	100	no	2916	7241	768	1920
IX	I81-I90	25	35	70	90	10	100	10	100	no	5210	13025	768	1920
X	I91-I100	30	40	90	95	10	100	10	100	no	3117	7709	768	1920

Table 10: First Class Instance Features.

name	row	var	lp	opt*	bound*	name	row	var	lp	opt*	bound*
I1	2688	2697	73230.21	62524	62524.00	I51	768	4948	81409.36	71998	71998.00
I2	2816	2825	76852.15	65046	65046.00	I52	896	5769	93495.19	81898	81898.00
I3	2944	2953	79598.69	67558	67558.00	I53	1024	6721	110137.25	97056	97056.00
I4	3072	3081	82467.66	70316	70316.00	I54	1152	7382	121747.65	107491	107491.00
I5	3200	3209	89923.52	76634	76634.00	I55	1280	8266	137308.63	120505	120505.00
I6	3328	3337	90672.21	77204	77204.00	I56	1408	9002	146998.74	129053	129053.00
I7	3456	3465	95919.27	81690	81690.00	I57	1536	9865	162489.16	142486	142486.00
I8	3584	3593	99262.82	84581	84581.00	I58	1664	10661	172075.23	151489	151489.00
I9	3712	3721	102724.65	87297	87297.00	I59	1792	11448	188109.13	165075	165097.00
I10	3840	3849	103892.85	88889	88889.00	I60	1920	12498	206427.59	182813	182813.00
I11	2688	4480	103802.28	88574	88574.00	I61	768	2071	24633.00	22044	22044.00
I12	2816	4749	112119.39	96366	96366.00	I62	896	2422	29081.00	26115	26115.00
I13	2944	4887	114630.26	97987	97987.00	I63	1024	2763	32215.00	29110	29110.00
I14	3072	5153	120447.69	103747	103747.00	I64	1152	3103	36243.00	32692	32692.00
I15	3200	5298	121641.43	103498	103498.00	I65	1280	3434	41155.00	37016	37016.00
I16	3328	5547	127386.90	108686	108686.00	I66	1408	3774	44071.00	39593	39593.00
I17	3456	5745	129721.92	112017	112017.00	I67	1536	4164	49400.00	44735	44735.00
I18	3584	5929	136534.29	116631	116631.00	I68	1664	4488	53344.00	48182	48182.00
I19	3712	6208	146245.82	125346	125346.00	I69	1792	4786	55799.00	50559	50559.00
I20	3840	6462	149534.57	128454	128454.00	I70	1920	5142	61016.00	54842	54842.00
I21	2688	4972	101348.05	87259	87259.00	I71	768	2916	46214.28	40982	40982.00
I22	2816	5161	104093.66	89548	89548.00	I72	896	3424	54264.62	47914	47914.00
I23	2944	5423	112511.36	96418	96418.00	I73	1024	3832	59692.33	52447	52447.00
I24	3072	5658	114270.26	98019	98019.00	I74	1152	4316	67701.80	59790	59790.00
I25	3200	5875	120846.51	104227	104227.00	I75	1280	4771	75300.45	66179	66179.00
I26	3328	6112	125679.74	107704	107704.00	I76	1408	5403	84719.96	75070	75070.00
I27	3456	6380	127368.59	109805	109805.00	I77	1536	5793	92447.25	81982	81982.00
I28	3584	6582	135650.71	116248	116248.00	I78	1664	6167	96766.53	85314	85314.00
I29	3712	6817	139600.57	119729	119729.00	I79	1792	6800	107506.65	95037	95037.00
I30	3840	7026	143834.45	123463	123463.00	I80	1920	7241	113069.40	100031	100031.00
I31	2688	6322	118147.88	102424	102424.00	I81	768	5210	80878.14	71426	71426.00
I32	2816	6546	120360.79	103159	103159.00	I82	896	6057	93438.94	82942	82942.00
I33	2944	6895	129601.72	111884	111884.00	I83	1024	6901	107806.34	96115	96115.00
I34	3072	7209	135934.57	117903	117903.00	I84	1152	7737	122734.53	110102	110102.00
I35	3200	7484	139207.04	120668	120668.00	I85	1280	8656	134801.82	119233	119233.00
I36	3328	7784	143078.51	123739	123739.00	I86	1408	9370	144003.97	128178	128178.00
I37	3456	8089	149990.22	130308	130308.00	I87	1536	10271	159046.15	142056	142056.00
I38	3584	8425	154962.59	133092	133092.00	I88	1664	11057	173756.92	154745	154770.00
I39	3712	8650	160788.47	138613	138613.00	I89	1792	11992	188147.70	167916	167916.00
I40	3840	8977	167495.03	144612	144612.00	I90	1920	13025	199612.81	176876	176916.00
I41	768	2071	35900.76	30866	30866.00	I91	768	3117	48477.88	42685	42685.00
I42	896	2422	41410.50	35771	35771.00	I92	896	3594	52366.41	46526	46526.00
I43	1024	2756	47323.86	40934	40934.00	I93	1024	4176	61537.48	54437	54437.00
I44	1152	3104	53126.97	46180	46180.00	I94	1152	4671	69087.68	60719	60719.00
I45	1280	3433	58235.01	50324	50324.00	I95	1280	5209	76853.31	68432	68432.00
I46	1408	3789	64249.15	55495	55495.00	I96	1408	5628	81652.19	72337	72346.00
I47	1536	4154	68933.49	59255	59255.00	I97	1536	6215	90864.53	80122	80122.00
I48	1664	4476	75464.66	65465	65465.00	I98	1664	6730	100490.40	88460	88460.00
I49	1792	4797	80288.60	69530	69530.00	I99	1792	7172	105321.35	92380	92380.00
I50	1920	5129	87439.72	75756	75756.00	I100	1920	7709	114205.89	100915	100915.00

Table 11: Second Class Instance Features.

name	row	var	lp	opt*	bound*	name	row	var	lp	opt*	bound*
U1	459	1000	50030.04	49797	49797.00	U51	445	1000	35329.47	34771	34774.00
U2	468	1000	49599.06	49490	49490.00	U52	467	1000	34412.77	33827	33838.75
U3	485	1000	49295.61	49020	49020.00	U53	464	1000	33787.34	33197	33216.00
U4	484	1000	49134.97	48972	48972.00	U54	443	1000	33549.08	32942	32955.33
U5	474	1000	50311.73	50149	50149.00	U55	456	1000	33949.75	33318	33325.88
U6	465	1000	49649.53	49466	49466.00	U56	457	1000	34120.88	33424	33465.54
U7	487	1000	50984.65	50666	50666.00	U57	437	1000	33971.65	33438	33440.50
U8	474	1000	50022.45	49859	49859.00	U58	435	1000	32683.93	32059	32113.75
U9	473	1000	50614.47	50358	50358.00	U59	464	1000	34452.02	33881	33887.50
U10	468	1000	50078.58	49961	49961.00	U60	447	1000	33566.19	32973	33010.36
U11	452	1000	46819.17	46266	46266.00	U61	433	1000	32004.41	31464	31496.89
U12	466	1000	46321.00	45628	45628.00	U62	460	1000	30905.62	30330	30335.20
U13	467	1000	46056.57	45531	45531.00	U63	439	1000	31239.41	30704	30723.65
U14	461	1000	45808.75	45218	45218.00	U64	453	1000	31876.25	31315	31353.36
U15	470	1000	46629.84	45978	45978.00	U65	448	1000	31733.75	31177	31186.00
U16	465	1000	46546.72	45795	45795.00	U66	477	1000	31608.03	31059	31067.50
U17	477	1000	47161.15	46471	46471.00	U67	444	1000	32401.24	31761	31834.68
U18	466	1000	46534.87	45877	45877.00	U68	448	1000	30933.08	30445	30459.71
U19	467	1000	47047.86	46356	46356.00	U69	473	1000	32663.75	32038	32066.45
U20	461	1000	46880.86	46217	46217.00	U70	439	1000	32135.82	31650	31705.00
U21	445	1000	42738.79	41946	41946.00	U71	439	1000	30916.17	30320	30487.62
U22	478	1000	42008.41	41346	41346.00	U72	456	1000	30915.38	30338	30495.40
U23	476	1000	41389.29	40694	40694.00	U73	444	1000	30457.51	29963	30051.37
U24	471	1000	41720.49	40955	40955.00	U74	438	1000	30088.56	29544	29626.54
U25	454	1000	41979.16	41235	41235.00	U75	465	1000	30346.40	29835	29973.79
U26	449	1000	41948.49	41168	41168.00	U76	439	1000	30741.84	30156	30319.61
U27	456	1000	42873.33	42054	42054.00	U77	431	1000	30319.63	29790	29928.42
U28	462	1000	42123.14	41475	41475.00	U78	456	1000	29930.41	29380	29479.91
U29	477	1000	43109.19	42277	42277.00	U79	453	1000	30179.69	29666	29790.09
U30	457	1000	42389.84	41684	41684.00	U80	437	1000	30341.32	29784	29935.58
U31	446	1000	39410.92	38685	38685.00	U81	409	1000	29969.06	29451	29586.24
U32	470	1000	38726.94	38106	38106.00	U82	451	1000	28709.27	28207	28307.75
U33	463	1000	38702.67	38067	38067.00	U83	452	1000	28299.20	27855	27979.38
U34	449	1000	37881.00	37159	37159.00	U84	447	1000	29947.00	29472	29560.59
U35	454	1000	38548.77	37826	37826.00	U85	441	1000	28811.23	28335	28453.58
U36	455	1000	38278.68	37488	37491.33	U86	451	1000	29104.73	28564	28699.96
U37	444	1000	39010.70	38237	38236.75	U87	452	1000	29090.14	28584	28718.29
U38	445	1000	38083.02	37372	37372.00	U88	452	1000	28961.51	28445	28562.54
U39	472	1000	39135.98	38374	38384.00	U89	437	1000	29495.70	29042	29144.33
U40	442	1000	38641.71	37965	37965.00	U90	430	1000	28415.19	27916	28061.62
U41	439	1000	36201.11	35538	35538.00	U91	441	1000	28209.62	27727	27872.82
U42	475	1000	35566.34	34934	34944.50	U92	467	1000	27122.92	26630	26771.13
U43	476	1000	35717.20	35071	35071.00	U93	450	1000	28277.61	27817	27916.20
U44	452	1000	36234.23	35596	35596.00	U94	449	1000	27700.70	27316	27387.50
U45	469	1000	35755.45	35112	35113.00	U95	462	1000	27596.56	27126	27251.84
U46	456	1000	35215.91	34529	34533.54	U96	446	1000	27980.33	27485	27623.50
U47	478	1000	36510.13	35866	35868.50	U97	475	1000	27484.81	27006	27111.65
U48	465	1000	35197.25	34564	34564.00	U98	462	1000	27371.23	26904	27000.10
U49	436	1000	36499.11	35883	35883.00	U99	444	1000	29277.14	28759	28877.14
U50	454	1000	35886.20	35311	35322.20	U100	438	1000	27714.50	27297	27385.89